COMP3026E Distributed Systems

10. Consistency Models

James Jianqiao Yu
Harbin Institute of Technology, Shenzhen

Consistency Models

- Contract between a distributed system and the applications that run on it
- A consistency model is a set of guarantees made by the distributed system

Linearizability

- All replicas execute operations in **some** total order

Linearizability

- All replicas execute operations in **some** total order
- That total order preserves the **real-time ordering** between operations

Linearizability

- All replicas execute operations in **some** total order
- That total order preserves the **real-time ordering** between operations
  - If operation A **completes** before operation B **begins**, then A is ordered before B in real-time

Linearizability

- All replicas execute operations in **some** total order
- That total order preserves the **real-time ordering** between operations
  - If operation A **completes** before operation B **begins**, then A is ordered before B in real-time
  - If neither A nor B completes before the other begins, then there is no real-time order
    - (But there must be some total order)

Real-Time Ordering Examples

P_A ├ w(x=1) ┤

P_B ├ w(x=2) ┤

P_C ├ w(x=3) ┤

P_D ├ w(x=4) ┤ ├ w(x=5) ┤

P_E ├ w(x=6) ┤

Linearizable?

P_A ├ w(x=1) ┤

P_B ├ w(x=2) ┤

P_C ├ w(x=3) ┤

P_D ├ w(x=4) ┤ ├ w(x=5) ┤

P_E ├ w(x=6) ┤

P_F ├ r(x)=1 ┤ ├ r(x)=2 ┤ ├ r(x)=3 ┤ ├ r(x)=6 ┤ ├ r(x)=5 ┤

Linearizable?

P_A ├ w(x=1) ┤

P_B ├ w(x=2) ┤

P_C ├ w(x=3) ┤

P_D ├ w(x=4) ┤ ├ w(x=5) ┤

P_E ├ w(x=6) ┤

P_F ├ r(x)=1 ┤ ├ r(x)=2 ┤ ├ r(x)=3 ┤ ├ r(x)=6 ┤ ├ r(x)=5 ┤ √

Linearizable?

P_A ├ w(x=1) ┤

P_B ├ w(x=2) ┤

P_C ├ w(x=3) ┤

P_D ├ w(x=4) ┤ ├ w(x=5) ┤

P_E ├ w(x=6) ┤

P_G ├ r(x)=1 ┤ ├ r(x)=2 ┤ ├ r(x)=5 ┤ ├ r(x)=6 ┤ ├ r(x)=5 ┤

Linearizable?

$P_A \vdash w(x{=}1) \dashv$

$P_B \quad \vdash w(x{=}2) \dashv$

$P_C \quad \vdash w(x{=}3) \dashv$

$P_D \quad \vdash w(x{=}4) \dashv \vdash w(x{=}5) \dashv$

$P_E \quad \vdash w(x{=}6) \dashv$

$P_G \vdash r(x){=}1 \dashv \vdash r(x){=}2 \dashv \vdash r(x){=}5 \dashv \vdash r(x){=}6 \dashv \vdash r(x){=}5 \dashv$ ✗

Linearizable?

P_A ├ w(x=1) ┤

P_B ├ w(x=2) ┤

P_C ├ w(x=3) ┤

P_D ├ w(x=4) ┤ ├ w(x=5) ┤

P_E ├ w(x=6) ┤

P_H ├ r(x)=1 ┤ ├ r(x)=4 ┤ ├ r(x)=2 ┤ ├ r(x)=3 ┤ ├ r(x)=6 ┤

Linearizable?

P_A ├ w(x=1) ┤

P_B ├ w(x=2) ┤

P_C ├ w(x=3) ┤

P_D ├ w(x=4) ┤ ├ w(x=5) ┤

P_E ├ w(x=6) ┤

P_H ├ r(x)=1 ┤ ├ r(x)=4 ┤ ├ r(x)=2 ┤ ├ r(x)=3 ┤ ├ r(x)=6 ┤ √

Linearizable?

P_A ├ w(x=1) ┤

P_B ├ w(x=2) ┤

P_C ├ w(x=3) ┤

P_D ├ w(x=4) ┤ ├ w(x=5) ┤

P_E ├ w(x=6) ┤

P_G ├ r(x)=1 ┤ ├ r(x)=4 ┤ ├ r(x)=5 ┤ ├ r(x)=6 ┤ ├ r(x)=3 ┤

Linearizable?

$P_A \vdash w(x=1) \dashv$

$P_B \qquad\qquad\qquad \vdash w(x=2) \dashv$

$P_C \qquad\qquad\qquad\qquad\qquad\qquad \vdash w(x=3) \dashv$

$P_D \qquad\qquad\qquad\qquad \vdash w(x=4) \dashv \vdash w(x=5) \dashv$

$P_E \qquad\qquad\qquad\qquad\qquad\qquad\qquad\qquad \vdash w(x=6) \dashv$

$P_G \vdash r(x)=1 \dashv \vdash r(x)=4 \dashv \vdash r(x)=5 \dashv \vdash r(x)=6 \dashv \vdash r(x)=3 \dashv$ ×

Linearizability = “Appears to be a Single Machine”

- Single machine processes requests one by one in order it receives them
  - Will receive requests ordered by real-time in that order
  - Will receive all requests in some order
- Atomic Multicast, Viewstamped Replication, Paxos, and RAFT provide Linearizability
- Single machine processing incoming requests one at a time also provide Linearizability

Linearizability is Ideal?

- Hides the complexity of the underlying distributed system from applications!
  - Easier to write applications
  - Easier to write correct applications

Linearizability is Ideal?

- Hides the complexity of the underlying distributed system from applications!
  - Easier to write applications
  - Easier to write correct applications
- But, performance trade-offs

Stronger vs Weaker Consistency

- Stronger consistency models
  √ Easier to write applications
  × More guarantees for the system to ensure
  - Results in performance tradeoffs
- Weaker consistency models
  × Harder to write applications
  √ Fewer guarantees for the system to ensure

Strictly Stronger Consistency

- A consistency model A is strictly stronger than B if it allows a strict subset of the behaviors of B
  - Guarantees are strictly stronger

Strictly Stronger Consistency

- A consistency model A is strictly stronger than B if it allows a strict subset of the behaviors of B
  - Guarantees are strictly stronger

$P_A \vdash w(x{=}1) \dashv$

$P_B \qquad\qquad \vdash w(x{=}2) \dashv$

$P_I \qquad\qquad\qquad\qquad \vdash r(x){=}? \dashv$

Strictly Stronger Consistency

- A consistency model A is strictly stronger than B if it allows a strict subset of the behaviors of B
  - Guarantees are strictly stronger

P_A ⊢ w(x=1) ⊣

P_B ⊢ w(x=2) ⊣

P_I ⊢ r(x)=? ⊣

r(x)= 2 1 0

Sequential Consistency

- All replicas execute operations in **some** total order
- That total order preserves the **process ordering** between operations
  - If process P issues operation A before operation B, then A is ordered before B by the process order
  - If operations A and B are done by different processes, then there is no process order between them
    - (But there must be some total order)

Sequential ≈ "Appears to be a Single Machine"

- Single machine processes requests one by one in order it receives them
  - Will receive requests ordered by process order in that order
  - Will receive all requests in some order

- The Single Machine is allowed to be **slow** or **delayed** relative to the real world

Linearizability Strictly Stronger Than Sequential

- Linearizability: $\exists$ total order + real-time ordering
- Sequential: $\exists$ total order + process ordering
  - Process ordering $\subseteq$ Real-time ordering

Sequential But Not Linearizable

A | ├ w(alicePwd=0x5f4) ┤

B | ├ r(alicePwd)=0x81d ┤

one week later

Consistency Hierarchy

Linearizability	e.g., RAFT
↓	
Sequential Consistency	
↓	
Causal+ Consistency	e.g., Bayou
↓	
Eventual Consistency	e.g., Dynamo

Causal+ Consistency

- Partially orders all operations, does not totally order them
  - Does not look like a single machine

- Guarantees
  - For each process, $\exists$ an order of all writes + that process's reads
  - Order respects the happens-before ($\rightarrow$) ordering of operations
  - + Replicas converge to the same state
    - Skip details, makes it stronger than eventual consistency

Causal Consistency

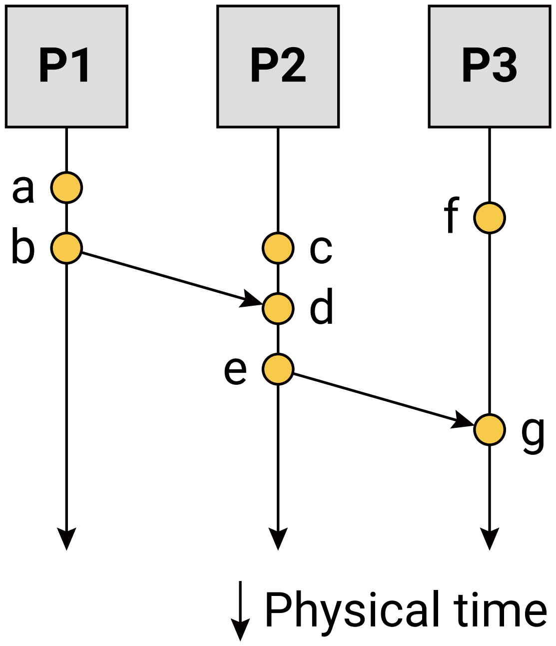
1. Writes that are **potentially** causally related must be seen by all processes in same order.

Causal Consistency

1. Writes that are **potentially** causally related must be seen by all processes in same order.
2. Concurrent writes may be seen in a different order on different processes. Concurrent: operations not causally related

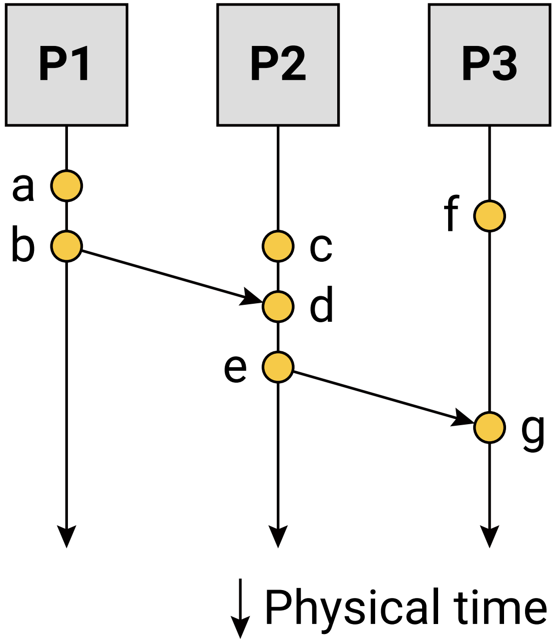
Causal Consistency

1. Writes that are **potentially** causally related must be seen by all processes in same order.
2. Concurrent writes may be seen in a different order on different processes. Concurrent: operations not causally related



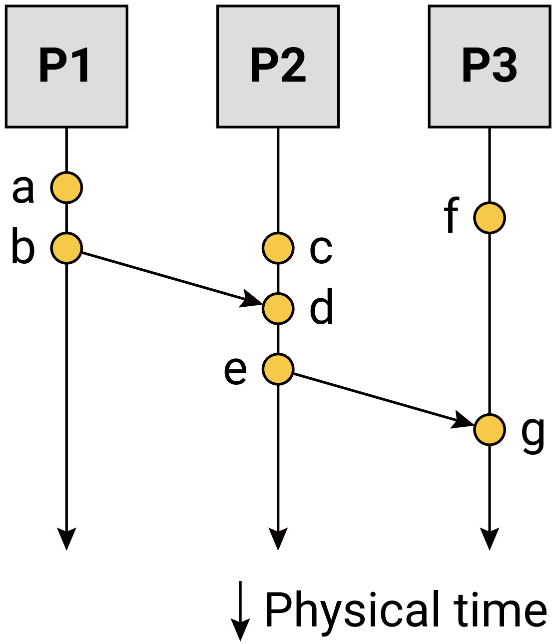
Causal Consistency

Operations	Concurrent?
a, b	



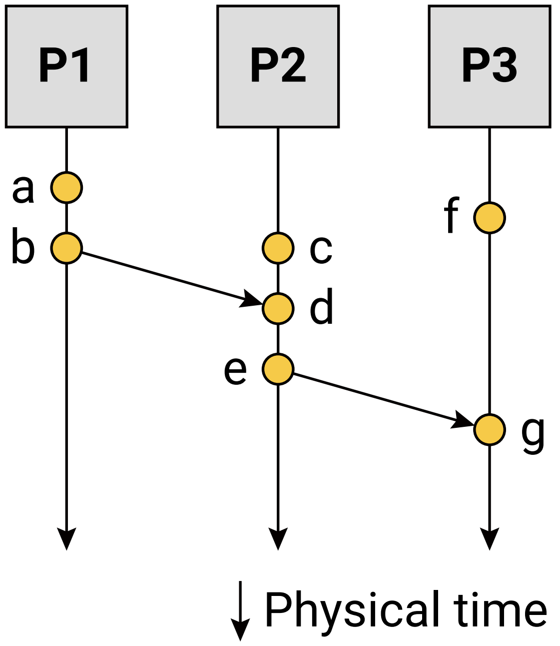
Causal Consistency

Operations	Concurrent?
a, b	N
b, f	



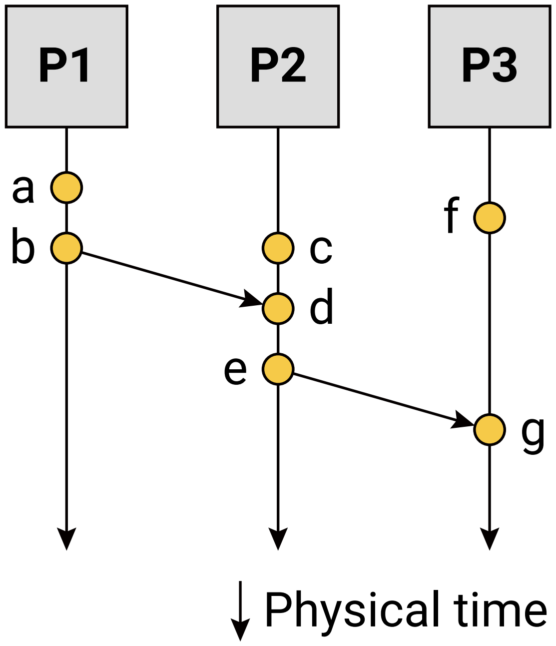
Causal Consistency

Operations	Concurrent?
a, b	N
b, f	Y
c, f	



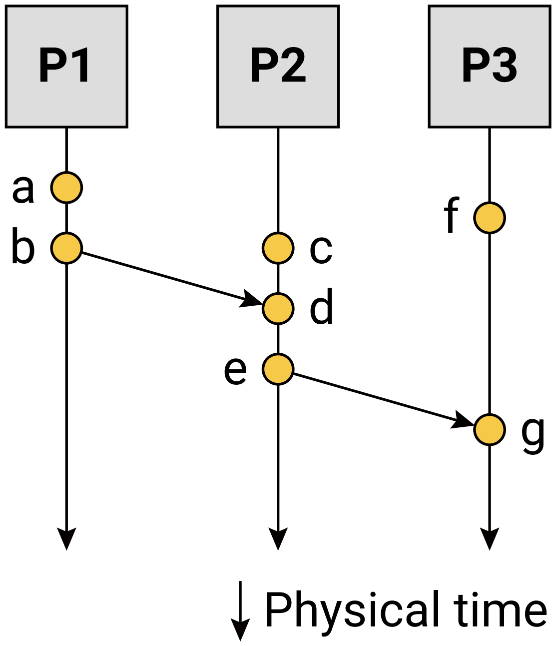
Causal Consistency

Operations	Concurrent?
a, b	N
b, f	Y
c, f	Y
e, f	



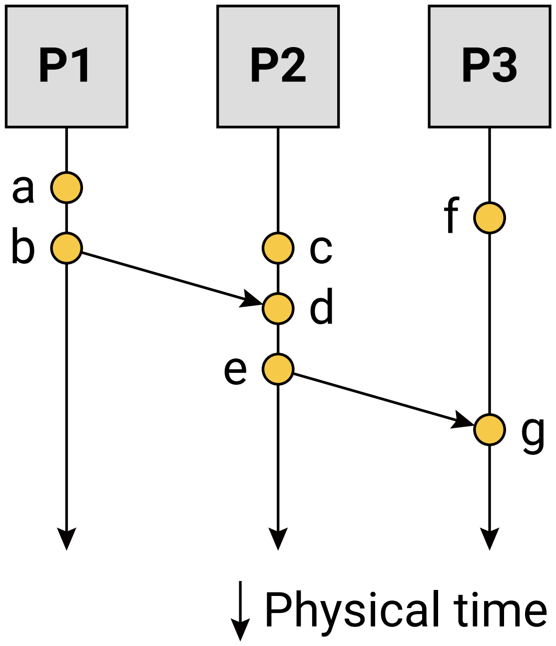
Causal Consistency

Operations	Concurrent?
a, b	N
b, f	Y
c, f	Y
e, f	Y
e, g	



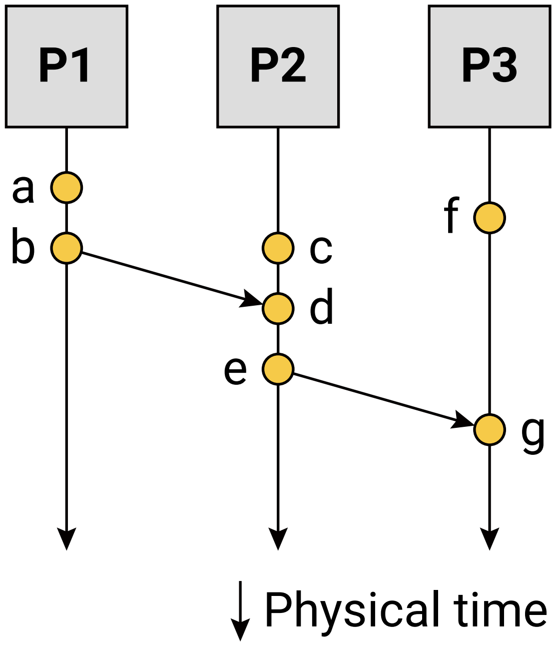
Causal Consistency

Operations	Concurrent?
a, b	N
b, f	Y
c, f	Y
e, f	Y
e, g	N
a, c	



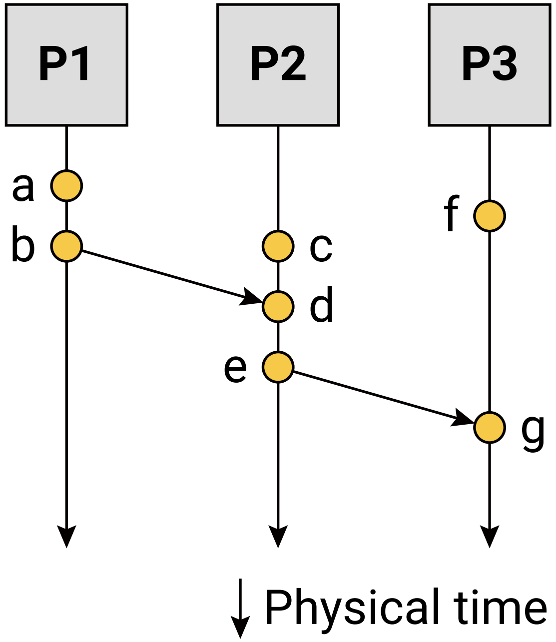
Causal Consistency

Operations	Concurrent?
a, b	N
b, f	Y
c, f	Y
e, f	Y
e, g	N
a, c	Y
a, e	



Causal Consistency

Operations	Concurrent?
a, b	N
b, f	Y
c, f	Y
e, f	Y
e, g	N
a, c	Y
a, e	N



Causal+ But Not Sequential

P_A ├ w(x=1) ┤ ├ r(y)=0 ┤

P_B ├ w(y=1) ┤ ├ r(x)=0 ┤

√ Causal+

Sequential ×

Causal+ But Not Sequential

P_A ├ w(x=1) ┤ ├ r(y)=0 ┤

P_B ├ w(y=1) ┤ ├ r(x)=0 ┤

√ Causal+

Happens-before order

w(x=1) -> r(y)=0

w(y=1) -> r(x)=0

Sequential ×

Causal+ But Not Sequential

P_A ├ w(x=1) ┤ ├ r(y)=0 ┤

P_B ├ w(y=1) ┤ ├ r(x)=0 ┤

√ Causal+

Happens-before order

w(x=1) -> r(y)=0

w(y=1) -> r(x)=0

P_A order: w(x=1), r(y)=0, w(y=1)

Sequential ×

Causal+ But Not Sequential

P_A ├ w(x=1) ┤ ├ r(y)=0 ┤

P_B ├ w(y=1) ┤ ├ r(x)=0 ┤

√ Causal+

Happens-before order
w(x=1) -> r(y)=0
w(y=1) -> r(x)=0

P_A order: w(x=1), r(y)=0, w(y=1)

P_B order: w(y=1), r(x)=0, w(x=1)

Sequential ×

Causal+ But Not Sequential

P_A ├ w(x=1) ┤ ├ r(y)=0 ┤

P_B ├ w(y=1) ┤ ├ r(x)=0 ┤

√ Causal+

Happens-before order
- w(x=1) -> r(y)=0
- w(y=1) -> r(x)=0

P_A order: w(x=1), r(y)=0, w(y=1)

P_B order: w(y=1), r(x)=0, w(x=1)

Sequential ×

Processing order
- w(x=1) -> r(y)=0
- w(y=1) -> r(x)=0

Causal+ But Not Sequential

P_A ├ w(x=1) ┤ ├ r(y)=0 ┤

P_B ├ w(y=1) ┤ ├ r(x)=0 ┤

√ Causal+		Sequential ×	
Happens-before order	w(x=1) -> r(y)=0 w(y=1) -> r(x)=0	Processing order	w(x=1) -> r(y)=0 w(y=1) -> r(x)=0
	P_A order: w(x=1), r(y)=0, w(y=1) P_B order: w(y=1), r(x)=0, w(x=1)	No total order	w(x=1) -> r(y)=0 w(y=1) -> r(x)=0

Causal+ But Not Sequential

P_A ├ w(x=1) ┤ ├ r(y)=0 ┤

P_B ├ w(y=1) ┤ ├ r(x)=0 ┤

√ Causal+		Sequential ×	
Happens-before order	w(x=1) -> r(y)=0 w(y=1) -> r(x)=0	Processing order	w(x=1) -> r(y)=0 w(y=1) -> r(x)=0

P_A order: w(x=1), r(y)=0, w(y=1)

P_B order: w(y=1), r(x)=0, w(x=1)

No total order

w(x=1) -> r(y)=0

w(y=1) -> r(x)=0

Causal+ But Not Sequential

$P_A \vdash w(x=1) \dashv \; \vdash r(y)=0 \dashv$

$P_B \vdash w(y=1) \dashv \; \vdash r(x)=0 \dashv$

√ Causal+

Happens-before order

w(x=1) -> r(y)=0

w(y=1) -> r(x)=0

P_A order: w(x=1), r(y)=0, w(y=1)

P_B order: w(y=1), r(x)=0, w(x=1)

Sequential ×

Processing order

w(x=1) -> r(y)=0

w(y=1) -> r(x)=0

No total order

w(x=1) -> r(y)=0

w(y=1) -> r(x)=0

Eventual But Not Causal+

P_A ├ w(x=1) ┤ ├ w(y=1) ┤

P_B ├ r(y)=1 ┤ ├ r(x)=0 ┤

√ Eventual

Causal+ ×

Eventual But Not Causal+

P_A ├ w(x=1) ┤ ├ w(y=1) ┤

P_B ├ r(y)=1 ┤ ├ r(x)=0 ┤

√ Eventual

As long as P_B eventually
would see r(x)=1
this is fine

Causal+ ×

Eventual But Not Causal+

P_A ├ w(x=1) ┤ ├ w(y=1) ┤

P_B ├ r(y)=1 ┤ ├ r(x)=0 ┤

√ Eventual

As long as P_B eventually would see r(x)=1 this is fine

Causal+ ×

Happens-before order

w(x=1) -> w(y=1)

r(y)=1 -> r(x)=0

Eventual But Not Causal+

P_A ├ w(x=1) ┤ ├ w(y=1) ┤

P_B ├ r(y)=1 ┤ ├ r(x)=0 ┤

√ Eventual

As long as P_B eventually would see r(x)=1 this is fine

Causal+ ×

Happens-before order

w(x=1) -> w(y=1)

r(y)=1 -> r(x)=0

No total order for P_B

w(x=1) -> w(y=1)

r(y=1) -> r(x)=0

Causal Consistency: Quiz

P_A ├ w(x=1) ┤ ├ w(x=3) ┤

P_B ├ r(x)=1 ┤ ├ w(x=2) ┤

P_C ├ r(x)=3 ┤ ├ r(x)=2 ┤

P_D ├ r(x)=2 ┤ ├ r(x)=3 ┤

- Valid under causal consistency

Causal Consistency: Quiz

P_A ⊢w(x=1)⊣ ⊢w(x=3)⊣

P_B ⊢ r(x)=1 ⊣ ⊢w(x=2)⊣

P_C ⊢ r(x)=3 ⊣ ⊢ r(x)=2 ⊣

P_D ⊢ r(x)=2 ⊣ ⊢ r(x)=3 ⊣

- Valid under causal consistency
- Why? $x = 3$ and $x = 2$ are concurrent
  - So all processes don’t (need to) see them in same order

Causal Consistency: Quiz

P_A ⊢ w(x=1) ⊣ ⊢ w(x=3) ⊣

P_B ⊢ r(x)=1 ⊣ ⊢ w(x=2) ⊣

P_C ⊢ r(x)=3 ⊣ ⊢ r(x)=2 ⊣

P_D ⊢ r(x)=2 ⊣ ⊢ r(x)=3 ⊣

- Valid under causal consistency
- Why? $x = 3$ and $x = 2$ are concurrent
  - So all processes don't (need to) see them in same order
- P_C and P_D read the values 1 and 2 in order as potentially causally related. No "causality" for 3.

Causal Consistency: Quiz

P_A ⊢ w(x=1) ⊣ ⊢ w(x=3) ⊣

P_B ⊢ r(x)=1 ⊣ ⊢ w(x=2) ⊣

P_C ⊢ r(x)=3 ⊣ ⊢ r(x)=2 ⊣

P_D ⊢ r(x)=2 ⊣ ⊢ r(x)=3 ⊣

- Invalid under sequential consistency

Causal Consistency: Quiz

P_A ⊢ w(x=1) ⊣ ⊢ w(x=3) ⊣

P_B ⊢ r(x)=1 ⊣ ⊢ w(x=2) ⊣

P_C ⊢ r(x)=3 ⊣ ⊢ r(x)=2 ⊣

P_D ⊢ r(x)=2 ⊣ ⊢ r(x)=3 ⊣

- Invalid under sequential consistency
- Why? P_C and P_D see 2 and 3 in different order
- But fine for causal consistency
  - 2 and 3 are not causally related

Causal Consistency: Quiz

P_A ⊦ w(x=1) ⊣

P_B ⊦ r(x)=1 ⊣ ⊦ w(x=2) ⊣

P_C ⊦ r(x)=2 ⊣ ⊦ r(x)=1 ⊣

P_D ⊦ r(x)=1 ⊣ ⊦ r(x)=2 ⊣

× $x = 2$ happens after $x = 1$

Causal Consistency: Quiz

P_A	$\vdash$ w(x=a) $\dashv$		
P_B		$\vdash$ w(x=b) $\dashv$	
P_C			$\vdash$ r(x)=b $\dashv$ $\vdash$ r(x)=a $\dashv$
P_D			$\vdash$ r(x)=a $\dashv$ $\vdash$ r(x)=b $\dashv$

√ P_B doesn't read value of 1 before writing 2

Copyright